

**Федеральное государственное автономное
образовательное учреждение высшего образования
«Национальный исследовательский университет
«Высшая школа экономики»**

**Факультет компьютерных наук
Основная образовательная программа
«Прикладная математика и информатика»**

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

**Программный проект на тему
«Сбор статистики
поступлений донатных
средств с помощью
генеративных моделей»**

**Выполнил студент группы 2211, 4 курса,
Павлов Кирилл Вадимович**

**Руководитель ВКР:
к. ф.-м. н., доцент, Лопаткин Виктор Евгеньевич**

**Соруководитель:
Щербина Александр Александрович**

Москва 2026

СОДЕРЖАНИЕ

СОДЕРЖАНИЕ.....	1
АННОТАЦИЯ.....	1
СПИСОК СОКРАЩЕНИЙ И ТЕРМИНОВ.....	2
ВВЕДЕНИЕ	2
Цель и задачи работы.....	3
Объект и предмет проектирования.....	4
Практическая значимость.....	4
ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ.....	4
1.1. Донаты как источник данных в стриминговой среде.....	4
1.2. Текст в видео и сложность его распознавания.....	5
1.3. Объектное детектирование для локализации донатных окон.....	5
1.4. OCR и vision-language модели.....	6
1.5. Ограничения существующих подходов и постановка задачи.....	6
ГЛАВА 2. ПРОЕКТИРОВАНИЕ СИСТЕМЫ.....	7
2.1. Общая архитектура пайплайна.....	7
2.2. Набор данных и разметка.....	8
2.3. Обучение детектора.....	8
2.4. Группировка детекций в события.....	9
2.5. Выбор модели для извлечения текста.....	10
ГЛАВА 3. РЕАЛИЗАЦИЯ ПРОТОТИПА.....	10
3.1. Организация входных и выходных данных.....	10
3.2. Детекция и сохранение стоп.....	11
3.3. Алгоритм объединения событий.....	11
3.4. Промпт для vision-language модели.....	12
3.5. Структура итогового JSON.....	12
3.6. Локальный запуск модели.....	12
ГЛАВА 4. ЭКСПЕРИМЕНТЫ И РЕЗУЛЬТАТЫ.....	13
4.1. Проверка качества детектора.....	13
4.2. Сравнение OCR и VLM.....	16

4.3. Анализ времени работы.....	17
4.4. Оценка итогового пайплайна	17
4.5. Ограничения проектной работы.....	18
ГЛАВА 5. ОПИСАНИЕ ПРИМЕНЕНИЯ ГЕНЕРАТИВНОЙ МОДЕЛИ.....	18
5.1. Генеративная модель как часть разрабатываемого продукта	18
5.2. Генеративная модель как инструмент разработки и подготовки текста.....	19
5.3. Разграничение ролей Qwen3-VL и ChatGPT	19
ПЕРСПЕКТИВЫ РАЗВИТИЯ ПРОГРАММНОГО РЕШЕНИЯ.....	20
6.1. Расширение набора данных.....	20
6.2. Оптимизация скорости.....	20
6.3. Улучшение поствалидации	20
6.4. Аналитический модуль.....	20
ЗАКЛЮЧЕНИЕ.....	21
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	21
ПРИЛОЖЕНИЕ А. СТРУКТУРА ФИНАЛЬНОГО ПАЙПЛАЙНА.....	22
ПРИЛОЖЕНИЕ Б. ДИНАМИКА ОБУЧЕНИЯ YOLO-ДЕТЕКТОРА.....	22

АННОТАЦИЯ

В работе рассматривается задача автоматического извлечения статистики о донатах из видеозаписей прямых трансляций. Донатные уведомления обычно отображаются как временные графические оверлеи поверх видеоряда и содержат имя отправителя, сумму, валюту и текст сообщения. Несмотря на то что такие данные визуальны присутствуют в публичных видеозаписях, они не представлены в машиночитаемом виде и недоступны внешнему исследователю через интерфейсы платформ. Это ограничивает независимый анализ монетизации, активности аудитории и коммуникационных паттернов в стриминговой среде.

Целью работы является разработка прототипа программного пайплайна, который обрабатывает видеозапись стрима, обнаруживает донатные окна с помощью модели объектного детектирования YOLO, объединяет повторяющиеся обнаружения одного и того же доната в единое событие, извлекает лучший crop уведомления и распознает его содержимое локальной vision-language моделью. Итоговая информация сохраняется в CSV и JSONL, что позволяет подсчитывать количество донатов, суммарные значения по валютам, анализировать тексты сообщений и проводить дальнейшую обработку данных.

В результате проекта подготовлен набор данных из 2900 размеченных скриншотов для 5 различных стримеров, обучена и протестирована модель детекции донатных блоков, реализован пайплайн группировки событий и проведено сравнение подходов к распознаванию текста. Первоначальный вариант с OCR оказался быстрым, но недостаточно точным для русскоязычных сообщений, вне зависимости от их сложности. После чего в финальной версии был применен локальный Qwen3-VL-8B-Instruct через llama.cpp server. Такой подход не требует внешних API, позволяет работать с данными локально и запускать прототип даже на слабом железе.

Ключевые слова: видеозаписи стримов, донаты, объектное детектирование, YOLO, OCR, vision-language model, Qwen3-VL, llama.cpp, структурированное извлечение данных, CSV.

СПИСОК СОКРАЩЕНИЙ И ТЕРМИНОВ

- API - интерфейс программного взаимодействия между приложениями
- BBox (bbox) - bounding box, прямоугольная рамка, ограничивающая объект на изображении
- Crop - вырезанная область изображения, содержащая найденный объект
- CVAT - инструмент разметки изображений и видео для задач компьютерного зрения
- OCR - optical character recognition, оптическое распознавание текста
- VLM - vision-language model, генеративная модель, способная обрабатывать изображение и текстовую инструкцию
- YOLO - семейство моделей объектного детектирования, выполняющих локализацию объектов на изображении

- mAP - mean average precision, усредненная метрика качества детектора
- VOD - video on demand, архивная видеозапись трансляции
- Overlay - текст, картинка или другой элемент, расположенный поверх основной потоковой трансляции
- Layout донатного уведомления – схема визуального расположения элементов донатных уведомлений. Под этим имеется ввиду имя донатера, сумма доната, валюта доната, текст доната

ВВЕДЕНИЕ

Индустрия прямых трансляций за последние годы стала важной частью цифровой экономики. Стриминговые платформы используются для развлечений, общения, освещения событий, проведения турниров, благотворительных акций и продвижения брендов. Одной из характерных форм монетизации стримов являются донаты - добровольные денежные переводы от зрителей в адрес стримера. Во многих случаях донат сопровождается текстовым сообщением и отображается на экране в виде визуального уведомления. Такое уведомление становится частью видеопотока: его видят зрители, оно записывается в архив трансляции и может быть изучено после завершения стрима.

Однако публичная видимость донатного уведомления не означает доступность данных для анализа. Для внешнего наблюдателя, исследователя или аналитика архивное видео остается единственным источником сведений. Ручное извлечение таких данных из длительных записей занимает много времени и плохо масштабируется. Поэтому возникает практическая задача: автоматически находить донатные уведомления в видеозаписях, распознавать их содержимое и сохранять результат в структурированном формате.

Технически эта задача находится на пересечении компьютерного зрения, обработки видео, распознавания текста и структурированного извлечения информации. Видеозаписи стримов содержат динамический фон, чат, веб-страницы, игровые интерфейсы, лицо стримера, элементы платформы и различные оверлеи. Донатные блоки могут иметь разные размеры, цветовые схемы, шрифты и анимации. Кроме того, они часто появляются на ограниченное время и повторяются в нескольких соседних кадрах. Если анализировать каждый кадр независимо, один донат может быть посчитан много раз. Поэтому полноценный продукт должен решать не только задачу детекции окна, но и задачу объединения временно близких обнаружений в одно событие.

В исходном проектном предложении задача формулировалась как двухэтапный пайплайн: сначала детектор YOLO локализует окно доната, затем OCR-модуль извлекает текстовые поля. В ходе экспериментов этот подход был уточнен. Детектор показал высокое качество и стал надежной основой системы, однако классический OCR оказался нестабильным на русскоязычных донатах с

цветными шрифтами, тенью, нестандартной компоновкой и длинными сообщениями. Поэтому финальная версия пайплайна использует локальную vision-language модель Qwen3-VL-8B-Instruct, которая получает стор донатного блока и возвращает структурированный JSON файл.

Актуальность работы определяется тем, что извлечение донатов из архивных видеозаписей позволяет получать независимые данные о публично видимой монетизации. Такие данные могут использоваться для исследовательских задач, анализа поведения аудитории, оценки динамики активности зрителей и проверки утверждений о доходах от стримов. При этом разработанная система не использует закрытые API платформ и не требует доступа к аккаунтам стримеров: она работает с тем, что уже отображено в видеоряде.

Цель и задачи работы

Цель работы - разработать и экспериментально проверить программный пайплайн для автоматического извлечения структурированной информации о донатах из видеозаписей стримов.

Для достижения цели были поставлены следующие задачи:

1. собрать и разметить набор изображений с положительными примерами донатных уведомлений и отрицательными кадрами без донатов;
2. обучить модель объектного детектирования для локализации донатных блоков в кадрах видеозаписи;
3. разработать алгоритм объединения повторяющихся детекций одного доната в единое событие;
4. сравнить подходы к распознаванию содержимого стор: классический OCR и локальную vision-language модель;
5. реализовать финальный пайплайн, который сохраняет обнаруженные данные донатов в виде таблиц для дальнейшего анализа;
6. оценить качество детектора и проанализировать ограничения распознавания текста

Объект и предмет проектирования

Объектом проектирования является процесс автоматического извлечения информации о донатах из архивных видеозаписей прямых трансляций. Предметом проектирования является программный пайплайн, включающий локализацию донатного уведомления в кадре, объединение повторяющихся детекций в одно событие, извлечение автора, суммы, валюты, времени и текста сообщения, а также экспорт результатов в машинно-читаемые таблицы.

Практическая значимость

Практическая значимость работы заключается в создании прототипа, который превращает визуально отображаемую информацию о донатах в таблицу. Такой результат можно использовать

для подсчета количества донатов за стрим, суммирования поступлений по валютам, поиска наиболее активных отправителей, анализа текстов сообщений и построения дополнительных метрик. Архитектура пайплайна также может быть адаптирована к другим задачам извлечения временных оверлеев из видео: рекламных плашек, игровых уведомлений, сообщений чата или статистических панелей.

ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1. Донаты как источник данных в стриминговой среде

Донатное уведомление в стриме является одновременно финансовым событием, элементом коммуникации и визуальным оверлеем. Содержательно оно обычно включает идентификатор отправителя, сумму, валюту и произвольный текст. В зависимости от используемого сервиса уведомление может сопровождаться изображением, видеофрагментом, звуком и анимацией. Для зрителей донат является публичным действием: он отображается на экране и становится прямым взаимодействием между аудиторией и стримером.

С точки зрения анализа данных донаты представляют интерес по нескольким причинам. Во-первых, они позволяют оценивать интенсивность финансовой поддержки во времени. Во-вторых, тексты сообщений отражают темы, шутки и реакции аудитории. В-третьих, динамика донатов может быть связана с событиями внутри трансляции: сменой темы разговора, появлением конкретного гостя, рекламной интеграцией или эмоциональным эпизодом. Однако для работы с такими данными требуется автоматизированный способ извлечения.

Существующие сервисы приема донатов ориентированы в первую очередь на владельца трансляции. Они предоставляют интерфейсы для отображения уведомлений и ведения внутренней статистики, но не решают задачу независимого ретроспективного анализа архивных видео. Это создает нишу для системы, которая работает не с закрытыми транзакционными данными, а с публичным визуальным представлением доната.

1.2. Текст в видео и сложность его распознавания

Распознавание текста в видео является отдельной задачей компьютерного зрения. В отличие от сканированного документа, текст на видеокадре может находиться на динамическом фоне, иметь нестандартный шрифт, тень, контур, низкое разрешение, цветовые эффекты и перспективные искажения. В обзоре Yin и соавторов задача видео-текста рассматривается как сочетание детекции, отслеживания и распознавания, где каждый этап влияет на итоговое качество [1]. Более новые обзоры также подчеркивают, что глубокие модели повысили качество распознавания, но реальные видео остаются сложной средой из-за малых размеров символов и неоднородного фона [2].

Донатные уведомления близки к классу *overlay text*, то есть текста, наложенного поверх исходного видеоряда. В работах по извлечению текстовых оверлеев из социальных видео показано, что такие элементы могут быть важным источником метаданных и часто требуют отдельной обработки [3]. В случае донатов сложность повышается за счет многоязычности, например сочетания кириллицы и латиницы, пользовательских никнеймов, эмодзи, пунктуации, разговорной лексики и нестандартной компоновки суммы.

1.3. Объектное детектирование для локализации донатных окон

Первый этап предлагаемой системы - локализация области кадра, в которой находится донатное уведомление. Для этой задачи подходят модели объектного детектирования, поскольку необходимо получить не только факт наличия доната, но и координаты прямоугольной области. Семейство YOLO широко используется для задач детекции благодаря высокой скорости работы и удобству обучения на пользовательских данных [5]. В данном проекте применялась модель YOLO26n, дообученная на собственном наборе кадров.

Выбор детектора вместо покадрового OCR по всему изображению обусловлен несколькими причинами. Во-первых, кадр стрима содержит множество лишних текстовых элементов: чат, элементы интерфейса, браузер, счетчики, надписи на видео. Во-вторых, OCR по всему кадру создает много шума и затрудняет выделение именно донатного текста. В-третьих, детектор позволяет вырезать компактный сгор, который затем можно передать отдельному модулю распознавания.

1.4. OCR и vision-language модели

Классические OCR-инструменты, такие как PaddleOCR, предназначены для выделения и распознавания текстовых областей на изображениях. PaddleOCR является открытым инструментарием для OCR-задач и поддерживает получение структурированного результата по изображению [8]. На раннем этапе проекта такой подход был реализован как отдельный прототип: для каждого сгора создавались варианты с увеличением разрешения, повышением контраста и резкости, после чего OCR-результаты оценивались эвристически.

Практические эксперименты показали, что OCR-подход дает быстрое, но нестабильное качество на донатах. Наиболее частые ошибки включали путаницу заглавных и строчных букв, замену кириллицы латиницей и цифры буквами, потерю пунктуации, неверное чтение суммы, а также затруднения при визуальном разбиении суммы на две строки. Предложенная реализация может использоваться в случаях, когда необходимо обработать много донатов за коротких срок, а качество исправить за счет человеческой проверки.

Vision-language модели решают задачу иначе: они получают изображение целиком и текстовую инструкцию, после чего могут вернуть структурированный ответ. Для текущей работы

это особенно полезно, потому что модель может учитывать layout: понимать, что верхняя строка до тире содержит автора и сумму, а нижний мелкий текст является сообщением. В финальном пайплайне использовалась Qwen3-VL-8B-Instruct в локальном запуске через llama.cpp server. На странице Qwen3-VL-8B-Instruct-GGUF приведены варианты локального запуска, включая запуск OpenAI-compatible сервера на базе llama.cpp [9]. Документация llama.cpp server также указывает возможность передачи изображений в поле image_url для мультимодальных моделей [10].

1.5. Ограничения существующих подходов и постановка задачи

Анализ предметной области показывает, что отдельные компоненты задачи хорошо изучены: существуют модели детекции объектов, OCR-инструменты, инструменты разметки и локальные генеративные модели. Однако в открытом виде не было обнаружено готового прикладного решения, которое извлекает донаты из архивных видеозаписей независимо от платформенного API. Поэтому основной вклад работы заключается не в создании нового нейросетевого алгоритма с нуля, а в инженерной интеграции нескольких подходов в единый воспроизводимый пайплайн.

Формально входом системы является видеозапись стрима. Выходом является таблица событий, где каждая строка соответствует одному донату и содержит время появления, координаты детекции, путь к сгор, автора, сумму, валюту, текст сообщения, полный видимый текст, confidence метрика анализа и флаг необходимости проверки. Для корректности результата система должна решать три подзадачи: локализовать донат, не считать одно уведомление несколько раз и извлечь текстовые поля из сгор.

ГЛАВА 2. ПРОЕКТИРОВАНИЕ СИСТЕМЫ

2.1. Общая архитектура пайплайна

Разработанная система имеет модульную архитектуру. Каждый модуль решает отдельную задачу и может быть заменен или улучшен без полной переработки остальных частей. Итоговая схема имеет следующий вид: видеозапись стрима → выбор кадров с заданным шагом → YOLO-детектор → сгор донатного блока с padding → объединение близких детекций в событие → выбор лучшего сгор → локальная vision-language модель → JSON → CSV/JSONL.

Модульность важна по двум причинам. Во-первых, качество детекции и качество распознавания текста имеют разные факторы ошибок. Детектор может успешно находить окно, даже если текст внутри него плохо читается. И наоборот, сильная языковая модель бесполезна, если ей передается неправильная область кадра. Во-вторых, проект развивается итеративно: первоначально после детектора использовался OCR, затем этот блок был заменен на VLM без необходимости заново обучать YOLO.

Таблица 1 - Основные этапы финального пайплайна

Этап	Вход	Выход	Назначение
Чтение видео	MP4 или другой видеофайл	последовательность кадров	получение кадров с учетом FPS и шага обработки
YOLO-детекция	кадр	bbox донатного блока	поиск области с донатом
Crop + padding	bbox и кадр	изображение доната	получение изображения для распознавания
Группировка	детекции соседних кадров	donation event	предотвращение повторного подсчета одного доната
VLM-извлечение	лучший crop события	JSON файл с полями доната	чтение автора, суммы, валюты и сообщения
Экспорт	JSON файлы событий	CSV/JSONL файлы	подготовка данных для анализа

2.2. Набор данных и разметка

Для обучения детектора был подготовлен пользовательский набор данных из 2900 скриншотов. В набор включались как positive-кадры с донатами, так и negative-кадры без донатов. Наличие отрицательных примеров важно, поскольку стриминговые кадры содержат множество визуальных элементов, которые могут быть похожи на донатные окна: чат, панели интерфейса, игровые уведомления, баннеры и элементы веб-страниц.

Разметка выполнялась в CVAT. CVAT является инструментом для аннотирования изображений и видео в задачах компьютерного зрения [7]. Для каждого положительного кадра размечалась прямоугольная область донатного блока. Экспорт выполнялся в формате Ultralytics YOLO Detection 1.0, что позволило напрямую использовать данные для обучения модели в инфраструктуре Ultralytics.

Разбиение на train, validation и test выполнялось не случайным перемешиванием отдельных изображений, а по группам «стример + дата». Такой подход снижает риск утечки близких кадров между обучением и тестом. Если кадры одного и того же стрима случайно попадают в разные части выборки, модель может получить слишком оптимистичную оценку, поскольку фон, layout и стиль уведомлений будут почти одинаковыми. Групповое разбиение делает тестирование более

реалистичным.

Таблица 2 - Разбиение набора данных

Часть набора	Всего кадров	Positive	Negative
train	1797	1005	792
validation	369	224	145
test	388	228	160

2.3. Обучение детектора

В качестве детектора использовалась модель YOLO26n. Обучение выполнялось с параметрами `imgsz=640`, `batch=16`, `epochs=50`. Размер изображения 640 был выбран как стандартный компромисс между скоростью и качеством детекции: донатный блок обычно занимает заметную часть кадра, но внутри него могут быть мелкие строки, поэтому слишком малый размер входного изображения нежелателен.

Для оценки детектора использовались метрики Precision, Recall, mAP50 и mAP50-95. Precision показывает долю корректных срабатываний среди всех предсказаний модели, Recall - долю найденных объектов среди всех истинных объектов. mAP50 оценивает среднюю точность при пороге IoU 0.50, а mAP50-95 является более строгой метрикой, усредненной по нескольким порогам IoU.

Таблица 3 - Итоговые метрики YOLO-детектора

Метрика	Значение на TEST
Precision	0.982
Recall	0.934
mAP50	0.968
mAP50-95	0.878

Полученные значения показывают, что детектор решает задачу локализации донатного окна достаточно надежно. Дополнительная проверка на реальных видеофрагментах показала, что модель корректно обрабатывала на разных случаях. При тестировании на видео с большим количеством донатов нашла все донаты, в видео с донатом плохого качества и с перекрытием друг друга модель не ошиблась, а на пустом фрагменте без донатов не дала ложных срабатываний. Иногда bbox немного обрезал края уведомления, поэтому в пайплайне используется padding при сохранении crop. Такой способ позволяет компенсировать небольшие ошибки локализации без изменения модели.

2.4. Группировка детекций в отдельные события

Донатное уведомление отображается на экране в течение нескольких секунд. При

покадровой обработке один и тот же донат будет обнаружен многократно: например, при шаге `frame_step=10` и FPS около 30 уведомление длительностью 5 секунд может дать около 15 срабатываний. Если каждое срабатывание считать отдельным донатом, итоговая статистика будет завышена. Поэтому после детекции необходима группировка.

В разработанном алгоритме каждая новая детекция сравнивается с уже существующими событиями. Детекция присоединяется к событию, если она достаточно близка по времени и пространственно похожа на последний `bbox` этого события. Используются три условия: максимальный временной разрыв, IoU между рамками и нормированное расстояние между центрами. Если подходящего события нет, создается новое.

Для каждого события сохраняются несколько лучших `score`-кандидатов. Кандидат оценивается по `confidence` детектора и относительной площади `bbox`. После завершения обработки видео для распознавания выбирается лучший `score` события. Такой подход снижает вычислительную нагрузку: VLM вызывается не для каждого кадра, а только один раз на каждое сформированное событие.

2.5. Выбор модели для извлечения текста

Первоначально после YOLO применялся OCR-подход. Он был реализован как рабочий быстрый прототип: `score` увеличивался, обрабатывался вариантами контраста и резкости, затем PaddleOCR возвращал текстовые блоки. Этот вариант полезен как `baseline` и может применяться для предварительного просмотра или грубого анализа, поскольку работает быстрее VLM. Однако его точность оказалась недостаточной для финального извлечения структурированных донатов.

После этого были протестированы локальные `vision-language` модели Qwen3-VL-4B-Instruct и Qwen3-VL-8B-Instruct. Целью тестирования было понять, насколько модель способна не только прочитать текст, но и правильно разнести его по полям JSON. Особенно важными были случаи с кириллицей, длинными сообщениями, отсутствующей валютой и суммой, визуально разбитой на две строки. По результатам ручного тестирования Qwen3-VL-8B-Instruct показала более надежное понимание `layout` с достаточным быстродействием и была выбрана для финального пайплайна.

Для отрезания зависимости от любых API, что поможет в работе в любых условиях доступа, было решено запускать модель на своем компьютере. Поэтому модель запускается локально через `Llama.cpp server`. Сервер предоставляет `OpenAI-compatible endpoint`, а изображение передается как `base64 data URL` в поле `image_url`. Это позволяет использовать единый клиентский код для обращения к локальной модели и сохраняет все данные на машине пользователя.

ГЛАВА 3. РЕАЛИЗАЦИЯ ПРОТОТИПА

3.1. Организация входных и выходных данных

Входными данными системы являются путь к видеофайлу, путь к весам YOLO-модели и параметры обработки. Основные параметры включают device, размер изображения для детектора, порог confidence, шаг обработки кадров, padding по осям X и Y, максимальный временной разрыв для группировки событий и адрес локального VLM-сервера.

Выходная директория содержит несколько типов файлов. Во-первых, сохраняются лучшие crop каждого найденного события и кадры с визуализированным bbox. Во-вторых, создается файл donations.jsonl, где каждая строка содержит сырой и распарсенный ответ модели для одного события. В-третьих, формируется events_summary.csv - главная таблица для анализа. Дополнительно создается totals_by_currency.csv, в котором донаты агрегируются по валютам. Из него очень удобно сразу подсчитывать все зафиксированные донаты за проанализированное видео.

Таблица 4 - Выходные артефакты пайплайна

Файл	Назначение
events_summary.csv	основная таблица событий: время, bbox, автор, сумма, валюта, сообщение, crop
donations.jsonl	полные JSON-ответы VLM и служебная информация по каждому событию
totals_by_currency.csv	агрегация количества донатов и суммы по валютам
run_metadata.json	параметры запуска: видео, модель, FPS, frame_step, количество детекций и событий
events/best_crops	изображения crop, отправленные в VLM
events/annotated_frames	кадры с нарисованными рамками детектора

3.2. Детекция и сохранение crop

При обработке видео скрипт открывает файл с помощью OpenCV и считывает кадры с заданным шагом. Для каждого выбранного кадра вызывается YOLO predict. Если модель возвращает одну или несколько рамок, каждая рамка приводится к границам изображения, расширяется на padding и используется для вырезания crop. В память события сохраняется не только crop, но и confidence, номер кадра, timestamp, исходный bbox и расширенный bbox.

Padding является важной практической деталью. На тестовых примерах bbox иногда немного обрезал верхнюю или нижнюю часть донатного окна. Это не мешало считать детекцию успешной, но могло ухудшить распознавание текста. Расширение рамки на несколько десятков пикселей

позволяет сохранить края уведомления, не добавляя слишком много лишнего фона.

3.3. Алгоритм объединения событий

Группировка событий реализована как онлайн-алгоритм. На каждом шаге новая детекция сравнивается с уже существующими событиями, у которых разница между текущим временем и временем окончания события не превышает `event_gap_sec`. Для каждого кандидата вычисляется IoU и нормированное расстояние между центрами `bbox`. Если IoU выше порога или расстояние меньше заданного порога, детекция присоединяется к наиболее похожему событию. В противном случае создается новое событие.

Такой подход не требует сложного трекинга и хорошо соответствует задаче, поскольку донатное окно обычно сохраняет близкую позицию на экране в течение времени показа. Если уведомление анимировано и немного сдвигается, условие по центру позволяет все равно объединить кадры. Если в кадре появляется новый донат после паузы, временной порог предотвращает объединение разных событий.

3.4. Промпт для vision-language модели

Для получения структурированного результата модель получает текстовую инструкцию и изображение стор. В инструкции явно задается схема JSON и правила интерпретации layout. Особое внимание уделяется разделению верхнего заголовка и нижнего сообщения: `donor` находится в верхнем заголовке до тире, `amount` - после тире, а `message` - ниже заголовка. Модель также получает правило для split-суммы: если первая строка содержит «Имя - 1», а следующая строка содержит «500 RUB», итоговая сумма должна быть интерпретирована как «1 500 RUB».

В ответе модель должна вернуть только валидный JSON без Markdown и пояснений. Основная схема включает поля `visible_lines`, `donor`, `amount_text`, `amount_number`, `currency`, `message`, `full_visible_text`, `confidence`, `needs_review` и `warnings`. Поле `visible_lines` оказалось особенно полезным: оно позволяет видеть, как модель разделила изображение на строки, и может использоваться в будущей версии для дополнительной валидации.

Важное решение финального пайплайна заключается в отказе от агрессивного постпроцессинга текста сообщения. После улучшения промпта модель стала правильно выделять автора, сумму и сообщение на тестовых примерах. При этом попытки автоматически исправлять отдельные слова часто портили оригинальный пользовательский текст. Поэтому текущая версия нормализует только технические поля для таблицы, но не переписывает содержимое переменной `message`.

3.5. Структура итогового JSON

Типовой ответ VLM имеет следующий вид:

```
{
  "visible_lines": ["Resaqw - 200!", "Полик ..."],
  "donor": "Resaqw",
  "amount_text": "200 RUB",
  "amount_number": 200,
  "currency": "RUB",
  "message": "Полик ...",
  "full_visible_text": "Resaqw - 200!\nПолик ...",
  "confidence": 0.98,
  "needs_review": false,
  "warnings": []
}
```

Такой формат удобен для последующего сохранения в CSV: простые поля переносятся в колонки, а массивы `visible_lines` и `warnings` могут сохраняться как JSON-строки. Для аналитики основными являются `donor`, `amount_number`, `currency`, `message`, `start_time` и `end_time`. Поле `full_visible_text` полезно при ручной проверке и сравнении с исходным `сгор`.

3.6. Локальный запуск модели

Для запуска Qwen3-VL-8B-Instruct использовался llama.cpp server. Модель загружалась в формате GGUF, отдельно подключался multimodal projector. Сервер запускался на локальном адресе, например `http://127.0.0.1:8081/v1/chat/completions`. Python-скрипт формировал base64-представление `сгор` и отправлял его в OpenAI-compatible endpoint. Параметр `temperature` устанавливался равным 0, чтобы минимизировать вариативность ответов.

Практическое время обработки одного `сгор` в тестах составило от 38 секунд до 1 минуты 44 секунд в зависимости от длины текста и сложности его обработки. Это существенно медленнее OCR, но приемлемо для локальной обработки, поскольку VLM вызывается один раз на событие, а не на каждый кадр. В случае необходимости скорость можно повышать за счет более мощного оборудования, меньшей квантованной модели, каскадного режима OCR → VLM или объединения нескольких событий в одно.

ГЛАВА 4. ЭКСПЕРИМЕНТЫ И РЕЗУЛЬТАТЫ

4.1. Проверка качества детектора

Главным экспериментальным результатом является качество YOLO-детектора на отложенной тестовой выборке. Значения `Precision=0.982` и `Recall=0.934` показывают, что модель редко ошибается на отрицательных кадрах и находит большинство донатных блоков. `mAP50=0.968` указывает на высокую способность модели локализовать окно при стандартном пороге IoU, а

mAP50-95=0.878 подтверждает, что рамки в большинстве случаев достаточно точны и при более строгой оценке.

Логи обучения за первые 44 эпохи показывают, что детектор быстро достиг высокого качества уже к 10-й эпохе, после чего метрики продолжали постепенно стабилизироваться. Лучшее значение mAP50-95 среди сохраненных эпох составило 0.932 на 43-й эпохе. На независимой тестовой выборке итоговая модель показала Precision=0.982, Recall=0.934, mAP50=0.968 и mAP50-95=0.878.

Таблица 5 - Ключевые показатели обучения и тестирования YOLO-детектора

Этап	Precision	Recall	mAP50	mAP50-95
Эпоха 1, validation	0.563	0.606	0.597	0.482
Эпоха 10, validation	0.926	0.937	0.981	0.855
Эпоха 26, validation	0.973	0.987	0.994	0.895
Эпоха 43, validation	0.962	0.956	0.990	0.932
Эпоха 44, validation	0.978	0.968	0.992	0.931
test split	0.982	0.934	0.968	0.878

Эпоха 10 показывает быстрый выход модели на рабочее качество, эпоха 43 дает лучшее mAP50-95 среди сохраненных логов, а test split отражает итоговую проверку на отложенной выборке.

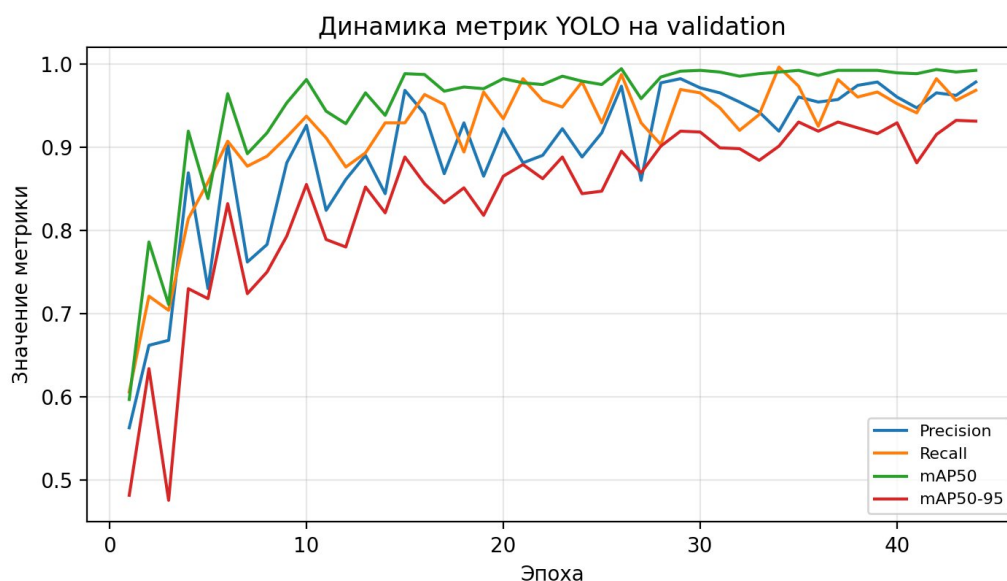


Рисунок 1 - Динамика метрик качества YOLO на validation за 44 эпохи



Рисунок 2 - Динамика функций потерь при обучении YOLO

Из графиков видно, что основное улучшение произошло в первой половине обучения, а на поздних эпохах модель работала в режиме уточнения границ и стабилизации качества. После 40-й эпохи Ultralytics отключил mosaic-аугментацию, что соответствует финальному этапу дообучения модели, и дообучил ее на изначальных кадрах.

Особенно важна проверка на реальном видео, которые ранее не были добавлены в базу данных, то есть были абсолютно новыми, потому что тестовая выборка из скриншотов не полностью отражает поведение пайплайна во времени. На 3 проверочных фрагментах модель обнаружила все донаты, не дала ложных срабатываний. Это позволило сделать вывод, что основная техническая проблема проекта сместилась с детекции к распознаванию содержимого спот.

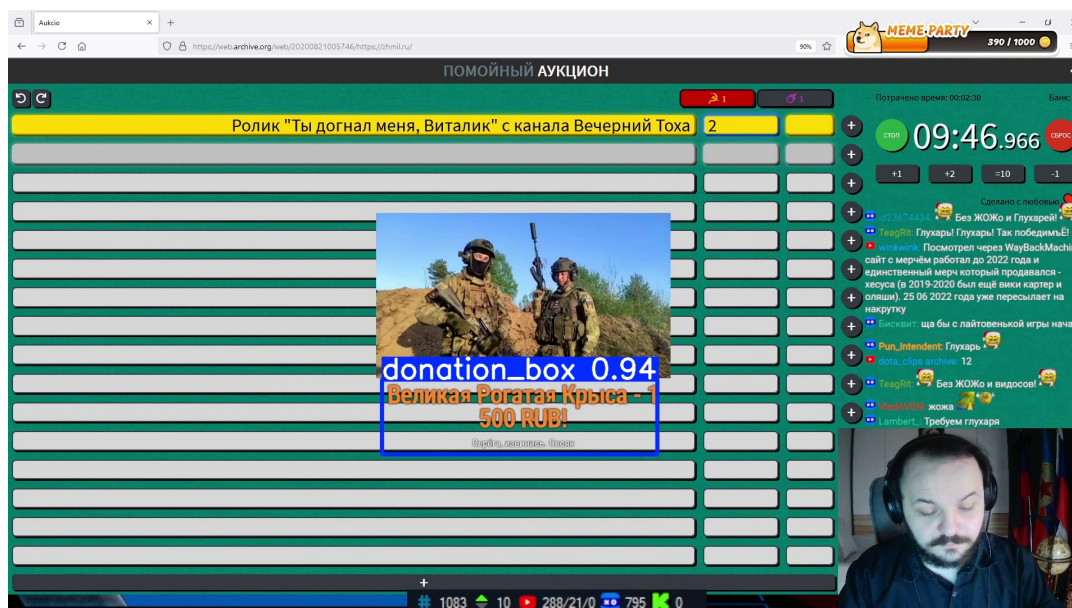


Рисунок 3 - Пример полного кадра с обнаруженным донатным блоком и confidence детектора 0.94

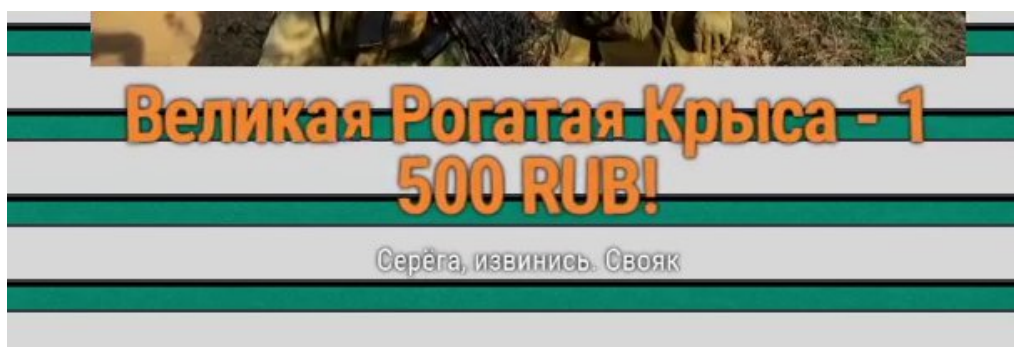


Рисунок 4 - Crop донатного блока со split-суммой «1» + «500 RUB»

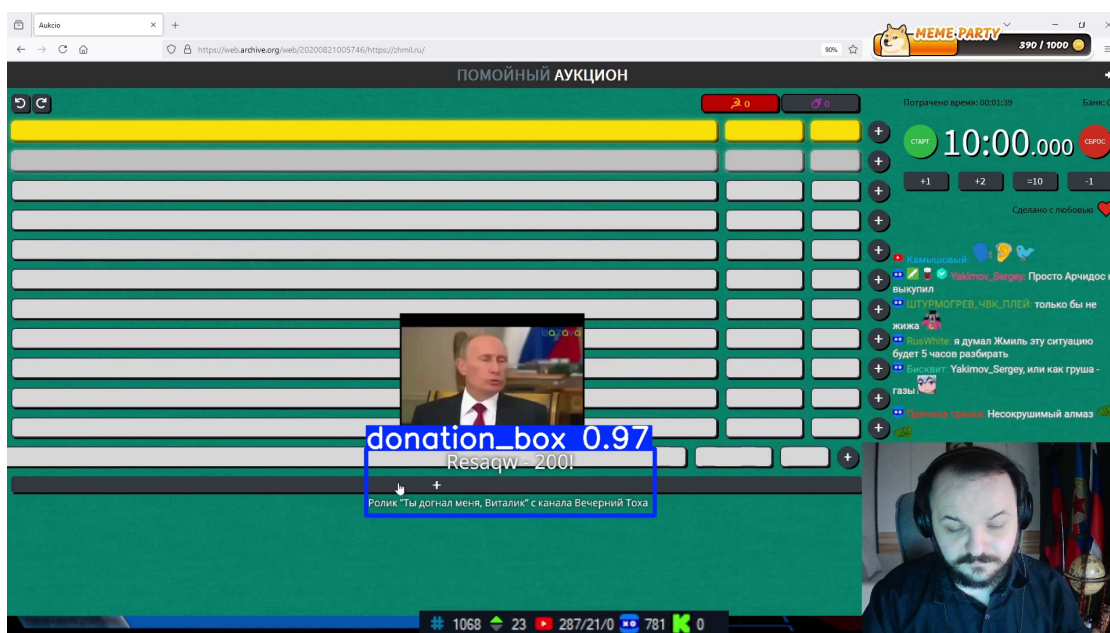


Рисунок 5 - Пример полного кадра с донатом, обнаруженным детектором с confidence 0.97

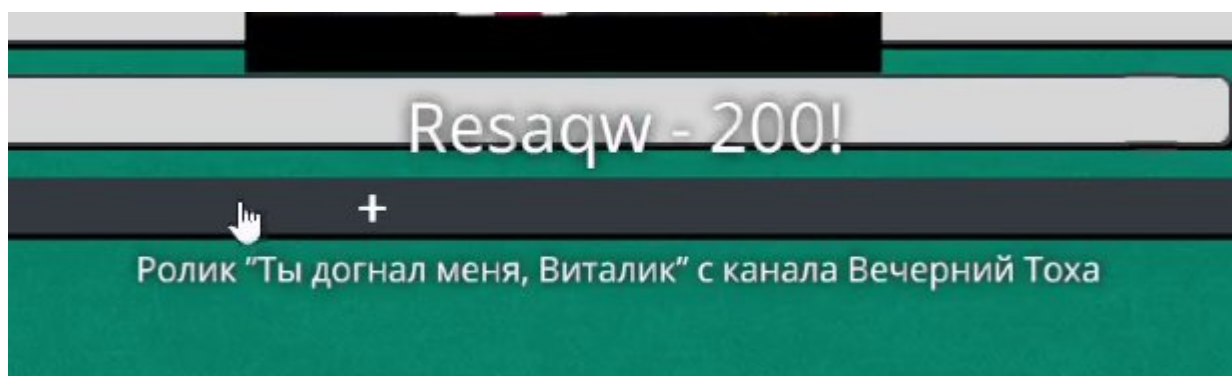


Рисунок 6 - Crop донатного блока с автором, суммой и текстовым сообщением

4.2. Сравнение OCR и VLM

OCR-прототип показал, что классическое распознавание текста может быстро давать

приблизительное содержание донатного сообщения, но его качество недостаточно стабильно для финальной таблицы. Основные ошибки возникали на цветном тексте с обводкой, мелких строках, смещении кириллицы и латиницы, а также при необычной компоновке суммы. Даже после увеличения изображения, повышения контраста и резкости большая часть ошибок сохранялась.

Vision-language модель Qwen3-VL-8B-Instruct намного лучше справилась с задачей структурного извлечения. После уточнения промпта модель корректно обработала три характерных примера: донат без валюты, донат со split-суммой 1 500 RUB и длинное русскоязычное сообщение. Особенно важным стало исправление второго случая: модель перестала воспринимать нижнюю строку сообщения как автора и правильно объединила «1» и «500 RUB» в amount_number=1500.

Таблица 6 - Результаты ручной проверки VLM на характерных примерах

Пример	Сложность	Результат после нового промпта
Resaqw - 200	нет валюты, короткое сообщение	автор, сумма и сообщение выделены корректно
Великая Рогатая Крыса - 1 / 500 RUB	сумма визуально разбита на две строки	получено amount_number=1500 и currency=RUB
Кок - 200 RUB	длинное сообщение с разговорной лексикой	структура выделена корректно, возможна ошибка в отдельном слове

Несмотря на улучшение, VLM не является идеальным OCR. В длинных сообщениях модель может заменить редкое или грубое слово на более вероятное с точки зрения языка. Это связано с генеративной природой модели: она не только распознает символы, но и использует языковой контекст. Поэтому поле confidence, которое возвращает сама модель, нельзя считать объективной метрикой точности. В текущей версии оно сохраняется как служебное поле, но окончательная оценка качества требует ручной валидации на размеченном наборе crop.

4.3. Анализ времени работы

Главный недостаток VLM-подхода - скорость. На тестовых изображениях Qwen3-VL-8B-Instruct обрабатывала один crop от 40 секунд до 1 минуты 44 секунд. Время зависело от длины сообщения и сложности изображения. Для онлайн-режима такая скорость недостаточна, но для ретроспективной офлайн-обработки она может быть приемлемой, особенно если число донатов в записи невелико по сравнению с числом кадров.

Ключевая оптимизация уже заложена в архитектуру: модель вызывается только для сгруппированного события, а не для каждого обнаружения. Например, если один донат отображался

10 секунд и был найден в 50 кадрах, VLM все равно вызывается один раз. В случае необходимости возможно добавить предварительную фильтрацию: короткие простые донаты обрабатывать OCR или более легкой моделью, а сложные случаи отправлять в Qwen3-VL-8B-Instruct.

4.4. Оценка итогового пайплайна

Проверка последней версии скрипта показала, что пайплайн не пропускает донаты на тестовом фрагменте без донатов. Это означает, что связка YOLO-детектора и группировки событий работает корректно на выбранных примерах. Финальная таблица содержит одну строку на донат, что позволяет использовать ее для аналитических операций: подсчета количества донатов, суммирования `amount_number` по валютам, фильтрации по донору, поиска сообщений и построения статистики по времени.

На текущем этапе качество системы можно охарактеризовать следующим образом. Детекция является сильной частью проекта: метрики высокие, а практическая проверка подтверждает надежность. Распознавание текста стало существенно лучше после перехода на VLM, но требует дальнейшей оценки на большем наборе `сгор`. Скорость работы остается ограничением, однако она не разрушает применимость решения для офлайн-анализа архивных записей.

4.5. Ограничения проектной работы

Работа имеет несколько ограничений. Во-первых, набор данных содержит 2900 скриншотов, что достаточно для прототипа, но может быть недостаточно для всех возможных стилей донатных уведомлений. Во-вторых, тестирование VLM проведено хоть и на различных важных, но не на всех возможных сложных примерах, ошибки все еще встречаются при работе на полных многочасовых стримах. В-третьих, скорость локальной модели зависит от аппаратного обеспечения и параметров квантования. В-четвертых, система обрабатывает только визуально видимые донаты и не может восстановить события, которые не отображались на экране или были закрыты другими элементами.

ГЛАВА 5. ОПИСАНИЕ ПРИМЕНЕНИЯ ГЕНЕРАТИВНОЙ МОДЕЛИ

5.1. Генеративная модель как часть разрабатываемого продукта

В финальной версии продукта генеративная `vision-language` модель используется для извлечения структурированных данных из `сгор` донатного блока. Название модели: Qwen3-VL-8B-Instruct. Адрес страницы модели: <https://huggingface.co/Qwen/Qwen3-VL-8B-Instruct-GGUF>. Способ применения: локальный запуск через `llama.cpp` server, передача изображения `сгор` и текстового промпта, получение JSON-объекта с полями `donor`, `amount_text`, `amount_number`, `currency`, `message`, `full_visible_text`, `confidence`, `needs_review` и `warnings`.

Цель применения данной модели - заменить недостаточно точный OCR-блок на более контекстно-осведомленный модуль, который способен учитывать расположение строк внутри донатного уведомления. В отличие от OCR, VLM получает инструкцию о структуре доната и может интерпретировать верхний заголовок как источник автора и суммы, а нижнюю строку как текст сообщения. Это особенно важно для случаев, где сумма визуально разбита на две строки или текст содержит смешение кириллицы, латиницы и пользовательской пунктуации.

Модель используется полностью локально. Это означает, что изображение стор не отправляется во внешние API. Такой режим выбран по двум причинам: во-первых, он повышает воспроизводимость экспериментов и позволяет работать без зависимости от коммерческого сервиса. Во-вторых, он снижает риски, связанные с передачей пользовательских данных третьей стороне.

5.2. Генеративная модель как инструмент разработки и подготовки текста

Отдельно в процессе выполнения работы использовалась генеративная модель ChatGPT. Название модели/сервиса: ChatGPT. Адрес сайта: <https://chatgpt.com>. Сервис OpenAI в виде AI-чатбота для решения задач, генерации идей и обучения [11].

Цели применения ChatGPT в рамках работы были следующими:

- помощь в исправлении и рефакторинге программного кода пайплайна;
- анализ ошибок OCR и VLM на примерах стор донатов;
- подготовка и улучшение промпта для Qwen3-VL;
- редактирование, перевод и стилистическая правка фрагментов текста ВКР;
- помощь в структурировании глав, таблиц и описания результатов.

Способ применения ChatGPT был интерактивным: модели передавались фрагменты кода, результаты экспериментов, изображения стор и черновые тексты. Модель предлагала варианты исправлений, формулировок и структуры. После этого проверялись предложенные изменения, исправлялся и проверялся локальный код, сравнивались результат с исходными изображениями и вносил собственные правки. Таким образом, генеративная модель использовалась как широкий вспомогательный инструмент разработки и редактирования.

5.3. Разграничение ролей Qwen3-VL и ChatGPT

В работе применялись две разные роли генеративных моделей. Qwen3-VL-8B-Instruct является частью программного продукта: она обрабатывает изображения донатов и возвращает структурированные данные. ChatGPT является инструментом поддержки разработки: она помогала исправлять код, улучшать промпт и редактировать текст.

ПЕРСПЕКТИВЫ РАЗВИТИЯ ПРОГРАММНОГО РЕШЕНИЯ

6.1. Расширение набора данных

Для повышения устойчивости системы возможно расширять набор данных до любой необходимой величины. Чем больше используется стримеров, платформ, разрешений видео, языков, цветовых схем и вариантов донатных уведомлений, тем лучше. Отдельно необходимо следить за сбором сложных случаев: частично перекрытые донаты, низкое разрешение, анимированные появления, светлый текст на светлом фоне, длинные сообщения и нестандартные валюты.

Чем больше проходит тестирований на реальных видео, тем качественней можно проанализировать текущую силу модели ли обнаружить возможные ошибки на редких сложных моментах.

6.2. Оптимизация скорости

Скорость VLM является главным ограничением текущей реализации. Возможные направления оптимизации включают использование более мощного GPU, тестирование других квантований, уменьшение max_tokens, уменьшение размера стоп без потери читаемости, каскадный режим OCR → VLM и параллельную обработку событий. Быстрый OCR-прототип можно использовать как вспомогательный режим, который дает приблизительный текст и может использоваться для предварительной фильтрации или ситуаций, где точность не критична.

6.3. Улучшение поствалидации

Хотя текущая версия сознательно не изменяет текст сообщения после VLM, технические поля можно проверять без риска исказить пользовательское сообщение. Например, amount_number должен быть числом, currency должна принадлежать известному набору валют, donor не должен быть пустым, а visible_lines должны содержать строки, соответствующие заголовку и сообщению. Есть возможность добавить мягкую валидацию, которая не изменяет message, а ставит флаг needs_review при подозрительных случаях и обрабатывать такие случаи по необходимости.

6.4. Аналитический модуль

После извлечения данных естественным развитием является аналитический модуль. Он может строить график донатов по времени, распределение сумм, рейтинг доноров, частотный анализ слов в сообщениях, сравнение активности между стримами и экспорт в BI-инструменты. Поскольку основной результат уже сохраняется в CSV, такая надстройка может быть реализована независимо от модуля компьютерного зрения.

ЗАКЛЮЧЕНИЕ

В ходе работы был разработан прототип системы для автоматического извлечения донатов из архивных видеозаписей стримов. Система решает задачу, которая практически важна для независимого анализа публично видимой монетизации: она не требует доступа к закрытым API и работает с видеорядом, сохраненным в записи трансляции.

Для локализации донатных блоков был собран и размечен набор из 2900 скриншотов, включающий положительные и отрицательные кадры. Разбиение выполнялось по группам «стример + дата», что снижает риск утечки похожих кадров между выборками. Обученная модель YOLO26n показала высокие метрики на тесте: Precision=0.982, Recall=0.934, mAP50=0.968 и mAP50-95=0.878. Практическая проверка на видео подтвердила, что детектор не пропускает донаты на тестовом фрагменте.

Первоначальный OCR-подход оказался полезным как быстрый прототип, но недостаточно точным для финального извлечения. Основные проблемы OCR возникали на кириллице, пунктуации, суммах, разбитых на строки, и длинных сообщениях. Поэтому в финальной версии использована локальная Qwen3-VL-8B-Instruct, запускаемая через llama.cpp server. Улучшенный промпт позволил модели корректно выделять автора, сумму, валюту и текст сообщения на характерных примерах.

Итоговый пайплайн выполняет полный цикл: видео → YOLO → группировка событий → лучший стор → локальная VLM → JSON → CSV. Полученные таблицы можно использовать для подсчета количества донатов, суммирования по валютам и дальнейшего анализа сообщений. Главным ограничением текущей версии остается скорость VLM и необходимость расширенной оценки на большем наборе стор. Тем не менее, результаты показывают, что предложенная архитектура является работоспособной и может служить основой для дальнейшего развития продукта.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] X.-C. Yin, Z.-Y. Zuo, S. Tian, and C.-L. Liu, “Text detection, tracking and recognition in video: A comprehensive survey,” IEEE Transactions on Image Processing, vol. 25, no. 6, pp. 2752-2773, Jun. 2016.
- [2] X.-F. Wang, Z.-H. He, K. Wang, Y.-F. Wang, L. Zou, and Z.-Z. Wu, “A survey of text detection and recognition algorithms based on deep learning technology,” Neurocomputing, vol. 556, art. no. 126702, Nov. 2023.
- [3] A. Słucki, T. Trzcinski, A. Bielski, and P. Cyrta, “Extracting textual overlays from social media videos using neural networks,” arXiv:1804.10687, 2018.
- [4] W. Wu, Y. Zhao, Z. Li, J. Li, M. Z. Shou, U. Pal, D. Karatzas, and X. Bai, “ICDAR 2023 Video Text

Reading Competition for Dense and Small Text,” arXiv:2304.04376, 2023.

- [5] Ultralytics, “Ultralytics YOLO,” Ultralytics Docs. [Online]. Available: <https://docs.ultralytics.com/>. [Accessed: May 12, 2026].
- [6] Ultralytics, “Train Custom Data,” Ultralytics Docs. [Online]. Available: <https://docs.ultralytics.com/modes/train/>. [Accessed: May 12, 2026].
- [7] CVAT.ai, “Computer Vision Annotation Tool.” [Online]. Available: <https://www.cvat.ai/>. [Accessed: May 12, 2026].
- [8] PaddlePaddle, “PaddleOCR,” GitHub repository. [Online]. Available: <https://github.com/PaddlePaddle/PaddleOCR>. [Accessed: May 12, 2026].
- [9] Qwen, “Qwen3-VL-8B-Instruct-GGUF,” Hugging Face. [Online]. Available: <https://huggingface.co/Qwen/Qwen3-VL-8B-Instruct-GGUF>. [Accessed: May 12, 2026].
- [10] ggml-org, “llama.cpp server README,” GitHub repository. [Online]. Available: <https://github.com/ggml-org/llama.cpp/blob/master/tools/server/README.md>. [Accessed: May 12, 2026].
- [11] OpenAI, “ChatGPT.” [Online]. Available: <https://chatgpt.com>. [Accessed: May 12, 2026].
- [12] DonationAlerts, “DonationAlerts - the streamer’s most important tool.” [Online]. Available: <https://www.donationalerts.com>. [Accessed: May 12, 2026].

ПРИЛОЖЕНИЕ А. СТРУКТУРА ФИНАЛЬНОГО ПАЙПЛАЙНА

В приложении приведено краткое алгоритмическое описание без включения полного программного кода. Полный код целесообразно размещать во внешнем репозитории GitHub или Google Colab, а в тексте ВКР указывать ссылку на него. Такой подход делает основной текст чище и облегчает проверку архитектуры.

Алгоритм:

1. Открыть видео и считать FPS.
2. Для каждого frame_idx с шагом frame_step:
 - 2.1. Выполнить YOLO predict.
 - 2.2. Для каждой рамки создать crop с padding.
 - 2.3. Найти существующее событие по time gap, IoU и расстоянию центров.
 - 2.4. Если событие найдено - обновить его, иначе создать новое.
3. Для каждого события выбрать лучший crop.
4. Отправить crop в локальный Qwen3-VL server.
5. Получить JSON и сохранить поля в events_summary.csv.
6. Сформировать totals_by_currency.csv и run_metadata.json.

Ссылка на Google Colab с обучением Yolo модели:

https://colab.research.google.com/drive/1rWx_uVzTxr41Kair-WS-5Feh-UNaezNK?usp=sharing

Ссылка на Google Colab с анализом видео с помощью Yolo модели:

https://colab.research.google.com/drive/1PQLlFkqwdnMP3jk6jfi2lP-RXLcJq_B3

ПРИЛОЖЕНИЕ Б. ДИНАМИКА ОБУЧЕНИЯ YOLO-ДЕТЕКТОРА

Таблица Б.1 - Значения функций потерь по эпохам

Epoch	box_loss	cls_loss	dfl_loss
1	1.1960	8.1500	0.007812
2	1.0370	4.0120	0.006467
3	1.0510	2.3170	0.006829
4	1.0060	1.4520	0.006580
5	0.9239	1.0800	0.006245
6	0.9229	0.8604	0.005821
7	0.8600	0.7254	0.005472
8	0.8451	0.6902	0.005320
9	0.8190	0.6528	0.005272
10	0.8228	0.6118	0.005094
11	0.8403	0.5863	0.005204
12	0.7852	0.5371	0.004937
13	0.7703	0.4964	0.004824
14	0.7459	0.4731	0.004302
15	0.7195	0.4635	0.004386
16	0.7260	0.4443	0.004510
17	0.7174	0.4303	0.004520
18	0.7027	0.4234	0.004317
19	0.6969	0.3964	0.004058
20	0.6581	0.3990	0.003830
21	0.6749	0.3911	0.004113
22	0.6691	0.3762	0.003986
23	0.6745	0.3679	0.003945
24	0.6444	0.3599	0.003835
25	0.6266	0.3514	0.003641
26	0.6160	0.3613	0.003627
27	0.6037	0.3469	0.003568
28	0.5989	0.3294	0.003466
29	0.6309	0.3491	0.003855
30	0.5759	0.3227	0.003395

31	0.5787	0.2953	0.003282
32	0.5630	0.2870	0.003184
33	0.5727	0.2962	0.003418
34	0.5500	0.2821	0.003207
35	0.5746	0.3093	0.003390
36	0.5404	0.2943	0.003072
37	0.5435	0.2883	0.003096
38	0.5499	0.2718	0.003074
39	0.5018	0.2540	0.002846
40	0.5121	0.2564	0.002817
41	0.4259	0.1889	0.002868
42	0.4040	0.1673	0.002774
43	0.3937	0.1833	0.002692
44	0.3780	0.1583	0.002535

Таблица Б.2 - Валидационные метрики по эпохам

Epoch	Precision	Recall	mAP50	mAP50-95
1	0.563	0.606	0.597	0.482
2	0.662	0.721	0.786	0.634
3	0.668	0.704	0.711	0.476
4	0.869	0.814	0.919	0.730
5	0.730	0.858	0.838	0.718
6	0.904	0.907	0.964	0.832
7	0.762	0.877	0.892	0.724
8	0.783	0.889	0.917	0.750
9	0.881	0.912	0.953	0.793
10	0.926	0.937	0.981	0.855
11	0.824	0.911	0.943	0.789
12	0.861	0.876	0.928	0.780
13	0.890	0.893	0.965	0.852
14	0.844	0.929	0.938	0.821
15	0.968	0.929	0.988	0.888
16	0.940	0.963	0.987	0.856
17	0.868	0.951	0.967	0.833
18	0.929	0.894	0.972	0.851
19	0.865	0.966	0.970	0.818

20	0.922	0.934	0.982	0.865
21	0.881	0.982	0.977	0.879
22	0.890	0.956	0.975	0.862
23	0.922	0.948	0.985	0.888
24	0.888	0.978	0.979	0.844
25	0.917	0.929	0.975	0.847
26	0.973	0.987	0.994	0.895
27	0.860	0.929	0.958	0.869
28	0.977	0.903	0.984	0.901
29	0.982	0.969	0.991	0.919
30	0.971	0.965	0.992	0.918
31	0.965	0.947	0.990	0.899
32	0.954	0.920	0.985	0.898
33	0.942	0.939	0.988	0.884
34	0.919	0.996	0.990	0.901
35	0.960	0.973	0.992	0.930
36	0.954	0.925	0.986	0.919
37	0.957	0.981	0.992	0.930
38	0.974	0.960	0.992	0.923
39	0.978	0.966	0.992	0.916
40	0.960	0.952	0.989	0.929
41	0.947	0.941	0.988	0.881
42	0.965	0.982	0.993	0.915
43	0.962	0.956	0.990	0.932
44	0.978	0.968	0.992	0.931

В таблице приведены сохраненные значения метрик и функций потерь за 44 эпохи обучения. Полные исходные логи обучения были перенесены в структурированный вид для последующего анализа динамики качества.